MEMBER B · DATA + FRONTEND INTEGRATION

Ingest, Visualize & *Connect.*

Your job starts as the critical path — Agmarknet data. Then you become the bridge: making the backend visible through React components and real data.

CLOCK	DATA STACK	FRONTEND	CHARTS
24 Hours	**Agmarknet CSV → Supabase**	**Next.js 14 · localhost:3000**	**Recharts**

MAPS
react-simple-maps

PROTOCOLS DATA PIPELINE SETUP GATE CORE FRONTEND WEBSOCKET UI INTEGRATION POLISH COMPONENT MAP

CRITICAL KNOWLEDGE · PROTOCOLS YOU MUST KNOW

Frontend ↔ Backend *communication contracts.*

You're the bridge. These protocols define exactly how your frontend talks to Member A's backend — and what breaks when they're wrong.

REST · FETCH() TO LOCALHOST:8000

All API calls use `fetch("http://localhost:8000/...")`. Set `NEXT_PUBLIC_API_URL=http://localhost:8000` in `.env.local`. Always include `Content-Type: application/json` for POST requests. Handle HTTP status codes properly: 200 = success, 422 = bad input (show validation message), 429 = rate limited (show "please wait"), 500 = backend error (fall back to canned data).

WEBSOCKET · REAL-TIME DEBATE STREAM

Connect to `ws://localhost:8000/ws` using a custom hook. Send `{"action":"start","run_id":"..."}`. Receive events: `{"stage":"optimist|pessimist|risk|mediator","data":{...}}`. Each event arrives ~2s apart (deliberate stagger). Render each agent card as it arrives — this creates the "agents arriving" theater effect. If WS fails, fall back to REST polling on `POST /analyze` with exponential backoff.

CORS · THE #1 INTEGRATION BUG

Your browser will block `localhost:3000 → localhost:8000` requests without proper CORS headers. This is Member A's responsibility on the backend, but you need to know the symptoms: `Access-Control-Allow-Origin` errors in browser console. If you see these, ping Member A immediately. Do NOT try to fix it with browser extensions — fix the backend CORS config.

DATA FORMAT · JSON CONTRACTS

Agree on JSON shapes with Member A at Hour 0. Key contracts: forecast returns `{dates[], yhat[], yhat_lower[], yhat_upper[], mape}`. Agent results return `{verdict, confidence, reasoning, data_citations[]}`. Profit returns `{line_items[], total_revenue, incremental_gain}`. Type these in `lib/api.ts` immediately.

CRITICAL: THE CANNED-DATA FALLBACK PATTERN

Every API call in `lib/api.ts` must check for `?demo=canned` in the URL. If present, skip the real API and return pre-built static JSON. This means you need two code paths: real API fetch and canned data import. Build the canned path FIRST — it lets you develop the UI without waiting for Member A's backend to be ready.

HOUR 0 CRITICAL PATH · AGMARKNET DATA INGEST

This is the *most time-sensitive task* in the entire hackathon.

Without data in Supabase, Prophet can't fit, agents can't reason, and the demo shows nothing. You unblock the entire team.

01	**Download Agmarknet CSVs** 4 crops × 5 Telangana mandis · 2022–present	CRITICAL
02	**Clean & normalize columns** commodity, mandi, modal_price, min_price, max_price, arrival_date	manual
03	**Flag anomalies** Prices = 0, duplicates, gaps > 7 days	script
04	**Upsert to Supabase** Run agmarknet_ingest.py · target: 5,000+ rows in price_records	GATE BLOCKER

PRIMARY SOURCE

`agmarknet.gov.in` — the official APMC price reporting portal. Select State: Telangana, Commodity: Tomato/Onion/Chili/Turmeric, Market: Warangal/Bowenpally/Nizamabad/Karimnagar/Nalgonda. Export as CSV. The site is notoriously slow and sometimes down.

FALLBACK SOURCES

`data.gov.in` — official open data portal, may have older snapshots. `github.com/akhilesh-k/agmarket` — community mirror of Agmarknet data. If the main site is down, these are your lifelines. Check both at Hour 0 before committing to one source.

THE 5 MANDIS — ALL MUST BE TELANGANA

Warangal (Warangal Urban), Hyderabad/Bowenpally (Medchal), Nizamabad, **Karimnagar** (replaced Kurnool — which is in Andhra Pradesh), Nalgonda. A judge from a Telangana farming family will instantly know if you use an AP mandi. Get this right.

01

HOUR 0-2 · SETUP + DATA INGEST

Data first. *Frontend scaffold second.*

- [ ] **CRITICAL: Download Agmarknet data immediately**

  This is your Hour 0 Job #1. Don't scaffold the frontend first. Don't write components. Open `agmarknet.gov.in` and start downloading CSVs for all 20 crop-mandi pairs. If the site is down, switch to fallbacks within 15 minutes. Every minute spent here unblocks Member A's Prophet fitting.

- [ ] **Run `agmarknet_ingest.py` to populate Supabase**

  Parse the CSVs, normalize column names, flag anomalies (price=0, duplicates), upsert to `price_records` table. Target: 5,000+ rows. Verify with a COUNT query. This gate-blocks the Hour 2 smoke test.

  `data/agmarknet_ingest.py`

- [ ] **Scaffold Next.js 14 project**

  `npx create-next-app@14 frontend --typescript --tailwind --app`. Create `.env.local` with `NEXT_PUBLIC_API_URL=http://localhost:8000`. Create route stubs: `app/page.tsx` (landing), `app/dashboard/page.tsx` (main farmer view), `app/fpo/page.tsx` (FPO dashboard).

- [ ] **Create `lib/api.ts` with typed interfaces**

  Define TypeScript interfaces for all API responses: `ForecastResponse`, `AgentResult`, `ProfitBreakdown`, `DebateEvent`. Create fetch wrappers with error handling and `?demo=canned` fallback. This is the contract between you and Member A.

  `lib/api.ts`

G1

HOUR 2 · SMOKE TEST GATE

Your gate check: *does data exist?*

YOUR SPECIFIC GATE CRITERIA

→ Supabase `price_records` has 5,000+ rows across 20 crop-mandi pairs

→ Karimnagar appears in the data — NOT Kurnool

→ Next.js runs on `localhost:3000` without errors

→ `lib/api.ts` has typed interfaces ready for integration

IF DATA INGEST FAILS AT THE GATE

Switch to manual CSV creation. Create 4 small CSV files with 100 rows each of realistic price data for Tomato/Warangal, Onion/Hyderabad, Chili/Karimnagar, Turmeric/Nizamabad. Import directly via Supabase dashboard. Imperfect data beats no data.

02

HOUR 2-6 · CORE FRONTEND COMPONENTS

Build the visible pieces. *Use canned data initially.*

- [ ] **Build `MandiSelector.tsx` + `CropSelector.tsx`**

  5 Telangana mandis: Warangal, Bowenpally, Nizamabad, Karimnagar, Nalgonda. 4 crops: Tomato, Onion, Chili, Turmeric. These are simple select components that drive every other component's data fetching.

  `components/MandiSelector.tsx` `components/CropSelector.tsx`

- [ ] **Build `PriceChart.tsx` with Recharts**

  Line chart showing: actual historical prices (solid line), Prophet yhat (dashed line), confidence bands (yhat_lower, yhat_upper as shaded area). X-axis: dates. Y-axis: ₹ per quintal. Start with hardcoded sample data, then swap in real API data when Member A's forecast endpoint works.

  `components/PriceChart.tsx`

Build `ProfitBreakdown.tsx` — the receipt card

Kirana receipt-style ₹ table. Each line item on its own row: revenue, transport cost, APMC fee (note: paid by buyer), labor, packaging, total, incremental gain. Clean typography, monospaced numbers right-aligned. This is one of the strongest demo moments.

`components/ProfitBreakdown.tsx`

Build `AgentDebate.tsx` — 4 agent cards

Port from DebateSection.jsx (AGENT_IITH reference). 4 cards: Optimist (green), Pessimist (red), Risk Analyst (orange), Mediator (blue). Each card shows: agent name, verdict badge (SELL_NOW/HOLD/WAIT), confidence %, reasoning text, data citations. Cards should animate in sequentially when WebSocket events arrive.

`components/AgentDebate.tsx`

First Prophet fit — Tomato/Warangal

While your components render with canned data, help Member A verify the first Prophet model fits correctly. Query `GET /forecast/Tomato/Warangal` and visually inspect the chart output. Does the forecast look reasonable? Are the confidence bands sensible? Report back.

⚡ DEPENDS ON MEMBER A'S FORECAST ENDPOINT

03

HOUR 6-10 · WEBSOCKET INTEGRATION + REAL DATA

Connect to real backend. *Swap out canned data.*

Build `lib/websocket.ts` — WebSocket hook

Custom React hook: `useDebateStream(runId)`. Connects to `ws://localhost:8000/ws`. Sends start action. Receives events and updates state for each agent card. Includes reconnection logic with exponential backoff. Falls back to REST polling if WS connection fails 3 times.

`lib/websocket.ts`

WEBSOCKET-HOOK-PATTERN

```
// WebSocket hook pattern for AgentDebate.tsx const { agents, status } = useDebateStream(runId); // agents = { optimist?: AgentResult, pessimist?: AgentResult, ... } // status = "connecting" | "streaming" | "complete" | "error" // Render each agent card as it arrives (staggered ~2s apart) // On "error" → fall back to REST: POST /analyze and render all at once
```

Animate agent card arrival

When a WebSocket event arrives for an agent, animate the card sliding in (fade + translate-y). The 2-second stagger between agents creates natural "agents arriving sequentially" theater. Judges see this as real-time AI deliberation, not latency. Add a subtle loading pulse for agents that haven't arrived yet.

Swap PriceChart to real API data

Replace hardcoded chart data with `GET /forecast/{crop}/{mandi}` responses. The forecast endpoint returns `{dates, yhat, yhat_lower, yhat_upper, mape}`. Display the rolling-origin MAPE value on the chart — but quote it as a range ("8–12%"), not a single precise number.

DEPENDS ON MEMBER A'S FORECAST ENDPOINT BEING LIVE

Wire ProfitBreakdown to real data

After `POST /analyze` returns a `run_id`, call `POST /profit/calculate` with the run_id, quantity, and distance. Render the line-item breakdown. Verify the numbers make sense — transport cost should scale with distance, APMC fee should be ~1.5% of revenue.

04

HOUR 10-14 · INTEGRATION SPRINT

Pair with Member A. *Fix everything that breaks.*

3 uninterrupted hours paired with Member A

This is where type mismatches, JSON format differences, and CORS issues surface. Work side-by-side. One screen showing the browser, one showing the terminal. Fix issues immediately. Do NOT defer bugs to "later" — later is the demo.

PAIRED SESSION WITH MEMBER A — CLEAR YOUR SCHEDULE

Common integration bugs to watch for

CORS: `Access-Control-Allow-Origin` errors → Member A fixes backend middleware. **Types:** API returns `confidence: "85"` (string) but you expect `confidence: 85` (number) → add parsing. **Nulls:** API returns `null` for optional fields → add null checks. **Dates:** API returns ISO strings, Recharts needs Date objects → convert in `api.ts`.

Build every error state and loading state

Every component needs 3 states: loading (skeleton/pulse), success (data rendered), error (friendly message + retry button). The demo should NEVER show a blank screen, a raw error trace, or a frozen UI. Use the `?demo=canned` fallback as the error recovery path.

HOUR 14 · FULL DRY-RUN

Complete demo walkthrough: select crop/mandi → see forecast chart → trigger debate → watch agents arrive → see profit breakdown. Both text and voice paths. Will have bugs — fix only show-stoppers.

05

HOUR 14-20 · POLISH + SECONDARY FEATURES

FPO dashboard. *Visual delight. Mobile-first.*

Build `ChoroplethMap.tsx` — FPO dashboard

Use `react-simple-maps` with `telangana.geojson`. Color districts by average price for selected crop. Tooltip on hover showing mandi name + current modal price. This is P2 — cut at Hour 16 if behind. Replace with a static screenshot in the pitch deck if cut.

`components/ChoroplethMap.tsx`

Visual polish — the demo moments

Chart draw-in animation (line traces from left to right). Receipt card slide-up animation. Agent card sequential reveal. Confidence bar fill animation. Micro-interactions on crop/mandi selection. These aren't decorations — they're what makes judges say "this feels polished."

Mobile-first responsive pass

The farmer's dashboard should work on a phone. Stack the cards vertically. Make the crop/mandi selectors full-width. Ensure the voice button is large and thumb-friendly. If a judge pulls out their phone and opens the URL, it should work.

Hour 20–24: Support rehearsals + final fixes

Keep the frontend running. Support Member C's timed rehearsals. Fix any visual glitches that surface during walkthroughs. Verify `?demo=canned` fallback renders correctly. Record backup video of the complete demo flow.

⚡ MEMBER C NEEDS STABLE FRONTEND FOR REHEARSAL

QUICK REFERENCE · YOUR COMPONENT MAP

Every component *you own.*

`MandiSelector.tsx`

5 Telangana mandis. Drives data for all other components.

`CropSelector.tsx`

Tomato · Onion · Chili · Turmeric. Paired with mandi selector.

PriceChart.tsx

Recharts line chart. Historical + forecast + confidence bands.

AgentDebate.tsx

4 agent cards. Sequential WebSocket animation. Verdict badges.

ProfitBreakdown.tsx

Kirana receipt-style ₹ table. Line-item cost breakdown.

VoiceButton.tsx

MediaRecorder → blob → API. Recording/loading/playing states.

ChoroplethMap.tsx

P2 · react-simple-maps + telangana.geojson. Cut if behind.

lib/api.ts + lib/websocket.ts

Typed fetch wrappers. WS hook. Canned-data fallback logic.